

ContractGuard User Manual

2026/04/30

Students: **Chunyang Wang** **21115214**
 Justin Siak **22449184**

Supervisor: **Dr. Graham Healy**

Contents

Introduction.....	3
System Requirements.....	4
Required Software.....	4
Supported Browsers.....	5
Local Network and Port Requirements.....	5
Internet Access Requirements.....	5
AI Feature Requirements.....	5
Docker Permissions and Environment.....	6
API Access Requirements.....	6
Installation Instructions.....	6
Before You Begin.....	6
Start ContractGuard with Docker Compose.....	6
Check That the Docker Services Are Running.....	8
Stop ContractGuard.....	8
First Launch and Account Creation.....	9
Installation Verification Checklist.....	9
Installation Notes.....	9
User Interface Overview.....	10
Header and Global Controls.....	10
Dashboard View.....	10
Workspace Summary.....	11
Upload Panel.....	11
Upload History Panel.....	11
Specification Details Workspace.....	12
Test Coverage and Generated Cases.....	13
Run Controls.....	14
Settings Modal.....	14
Chat Panel.....	17
Logs View.....	17
Run Detail and AI Detail Windows.....	18
Using ContractGuard.....	18
Sign In and Open the Workspace.....	18
Upload an API Specification.....	19
Select an Uploaded Specification.....	19
Review the Parsed Specification.....	20
Review and Edit Generated Test Cases.....	20
Configure a Full Test Run.....	21
Base URL.....	21
Authentication.....	21

Run the Full Suite.....	22
Run a Single Test Case.....	22
Understand the Run Summary.....	23
Inspect Failed Cases.....	23
Use AI Summary for Failed Cases.....	24
Generate an Additional Test from a Failure.....	24
Use the Grounded Chat Panel.....	25
Review Run History in Logs.....	26
Clear All Logs.....	26
Clear Upload History.....	27
Github Action Integration.....	27
Quick Start in Github.....	27
Step 1: Create Workflow File.....	27
Step 2: Add GitHub Secrets.....	28
Step 3: Repository Structure.....	29
Step 4: Runtime Mapping File.....	30
Step 5: How Pipeline Works.....	30
Workflow Configuration Options.....	31
Example Configurations.....	31
Viewing Results.....	33

Introduction

ContractGuard is a web application for API contract testing. It allows a user to upload an OpenAPI or Swagger specification, generate test cases from that specification, run those tests against a live API, and review the results in a single workspace. The application also includes optional AI-assisted tools for failure explanation, follow-up test generation, and grounded chat tied to the selected specification.

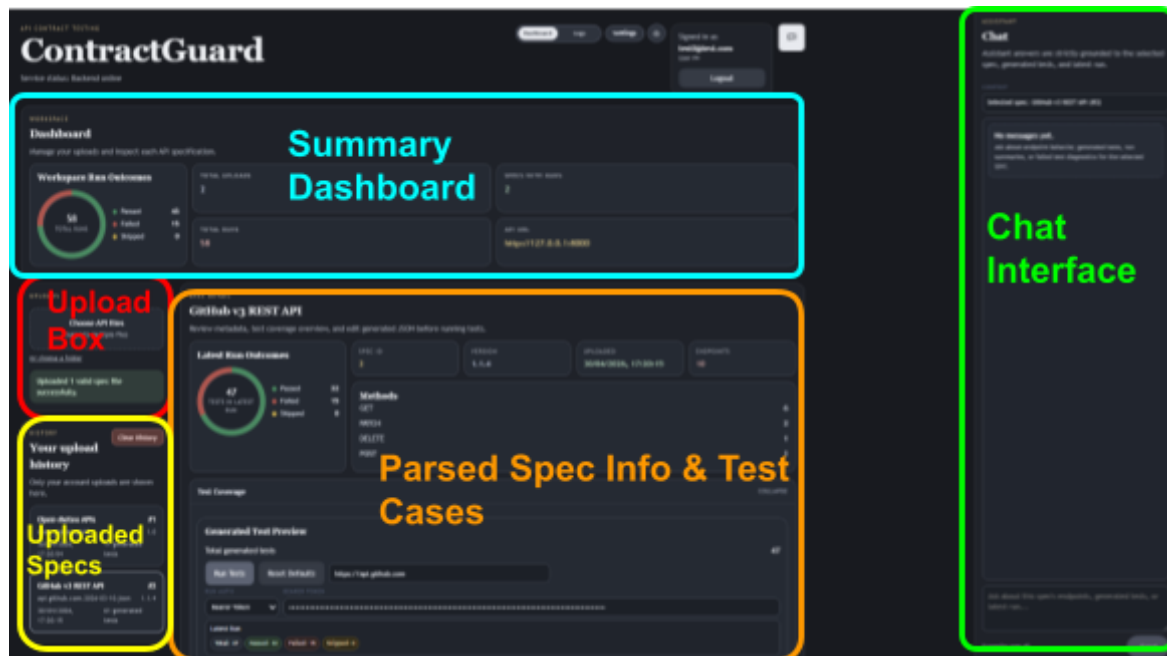
ContractGuard is intended for testers, developers, QA teams, and project teams that need a practical way to validate whether an implementation matches its published API contract. It is especially useful when a team wants to check required fields, status codes, common error paths, authentication behavior, and endpoint coverage without writing each test by hand before the first run.

The standard ContractGuard workflow is:

1. Start the application.
2. Create an account or sign in.
3. Upload one or more OpenAPI specification files.
4. Review the parsed API information and generated test cases.
5. Provide the base URL of the API under test.
6. Add authentication details if the API requires them.
7. Run the generated tests.
8. Inspect passed, failed, and skipped cases.

9. Use AI tools to explain failures or propose additional follow-up tests.
10. Review past runs in the Logs view.

This manual covers installation, first launch, user interface navigation, day-to-day testing workflows, AI setup, result interpretation, and troubleshooting.



ContractGuard Homepage

System Requirements

ContractGuard can be run on Windows, macOS, or Linux. The repository supports two practical ways to run the application:

Required Software

Component	Requirement
Docker Desktop or Docker Engine	Required
Docker Compose	Required
Web browser	Required

ContractGuard's supported release setup does not require Python, Node.js, npm, or PostgreSQL to be installed directly on the host machine. Those runtime components are provided inside the Docker containers started by Docker Compose.

Supported Browsers

Use a current version of a modern browser such as Microsoft Edge, Google Chrome, or Mozilla Firefox. The frontend is a Vite-based React application and is intended to be accessed in a normal desktop browser.

Local Network and Port Requirements

The default ports used by ContractGuard are:

- Frontend: 5173
- Backend API: 8000
- PostgreSQL in Docker Compose: 5432

These ports must be available on the machine running Docker so the containers can expose the ContractGuard frontend, backend, and database services correctly.

You access the application through:

- `http://localhost:5173`

You can verify the backend through:

- `http://localhost:8000/api/health`

Internet Access Requirements

Internet access is not required to start the core Docker stack itself once the necessary images are available locally. Internet access is required when:

- You test a remote API rather than a local one.
- You use cloud-based AI providers such as OpenAI or Anthropic.
- You need to browse the Ollama model library online.

AI Feature Requirements

AI features are optional. ContractGuard can run without any AI provider configured. If you want to use AI summaries, suggested test cases, or grounded chat with a configured model, you need one of the following:

- A local Ollama installation, typically available at `http://localhost:11434`
- An OpenAI API key
- An Anthropic API key

For OpenAI and Anthropic, ContractGuard requires an API key before it can load or use provider models. For Ollama, ContractGuard accepts a model name directly and uses the configured base URL, or the default Ollama URL if no base URL is entered.

If you use Ollama models with a `:cloud` suffix, access may fail unless Ollama cloud access is configured on the machine. In that case, either configure cloud access correctly or switch to a locally available Ollama model.

Docker Permissions and Environment

The user running ContractGuard must be able to start containers and bind the required local ports. Docker must be running before the application is started.

API Access Requirements

To run tests successfully, you need:

- A valid absolute API base URL such as `https://api.example.com`
- An API implementation that is reachable from the machine running ContractGuard
- A Bearer token or API key when the target API requires authentication

The current web interface requires the base URL before a run can start. If the target API is protected and you do not provide valid credentials, some tests will fail or be skipped depending on the endpoint behavior.

Installation Instructions

This section documents the supported release installation path for ContractGuard: Docker Compose.

Before You Begin

Open a terminal in the repository root folder before running any commands. All commands in this manual assume that your current working directory is the top level of the ContractGuard repository.

Ensure Docker Desktop or Docker Engine is running before you start the services.

On the first run, Docker may need to download base images and build the frontend and backend containers. This can take longer than later startups.

Start ContractGuard with Docker Compose

Docker Compose starts all required services for you:

- PostgreSQL database
- FastAPI backend
- Vite frontend

From the repository root, run one of the following commands.

PowerShell, Command Prompt, macOS Terminal, or Linux shell:

Shell

```
docker compose up --build
```

What this does:

- Builds the backend image from Dockerfile
- Builds the frontend image from Dockerfile
- Starts PostgreSQL 16 with a persistent Docker volume
- Starts the backend on port 8000
- Starts the frontend on port 5173

The Docker configuration supplies these development settings automatically:

- Database URL:
postgres+psycpg://contractguard:contractguard@db:5432/contractguard
- JWT secret: dev-secret-change-me
- Backend Python path: /app/src
- Frontend API base URL: <http://localhost:8000>

Wait until the containers finish building and the services are running before opening the application in your browser.

After the services start, open the application at:

- <http://localhost:5173>

To verify that the backend is healthy, open:

- <http://localhost:8000/api/health>

The health endpoint should return:

JSON

```
{"ok": true}
```

```

PS C:\FYP\2026-csc1097-wangc9-siakj2> docker compose up --build
✓ Service frontend Built 1.2s
✓ Container contractguard-db Created 0.0s
✓ Container contractguard-frontend Recreated 0.3s
✓ Container contractguard-backend Recreated 0.2s
Attaching to contractguard-backend, contractguard-db, contractguard-frontend
contractguard-db | PostgreSQL Database directory appears to contain a database; Skipping initialization
contractguard-db |
contractguard-db | 2026-04-30 14:26:08.148 UTC [1] LOG: starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
contractguard-db | 2026-04-30 14:26:08.148 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
contractguard-db | 2026-04-30 14:26:08.148 UTC [1] LOG: listening on IPv6 address ":::", port 5432
contractguard-db | 2026-04-30 14:26:08.152 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
contractguard-db | 2026-04-30 14:26:08.157 UTC [29] LOG: database system was shut down at 2026-04-29 23:48:15 UTC
contractguard-db | 2026-04-30 14:26:08.166 UTC [1] LOG: database system is ready to accept connections
contractguard-frontend |
contractguard-frontend | > contractguard-frontend@0.1.0 dev
contractguard-frontend | > vite --host 0.0.0.0 --port 5173
contractguard-frontend |
contractguard-frontend | VITE v7.3.2 ready in 282 ms
contractguard-backend | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
contractguard-db | 2026-04-30 14:31:08.246 UTC [27] LOG: checkpoint starting: time
contractguard-db | 2026-04-30 14:31:08.257 UTC [27] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.003 s, sync=0.002 s, total=0.011 s; sync files=2, longest=0.001 s, average=0.001 s; distance=0 kB, estimate=0 kB; lsn=0/1B1D008, redo lsn=0/1B1CFD0

```

Check That the Docker Services Are Running

If you want to confirm that the services started correctly from the command line, run:

```

Shell
docker compose ps

```

You should see running containers for the database, backend, and frontend.

If one of the services did not start correctly, inspect its logs with:

```

Shell
docker compose logs backend

```

```

Shell
docker compose logs frontend

```

```

Shell
docker compose logs db

```

Stop ContractGuard

To stop the running services, press Ctrl+C in the terminal where Compose is running.

To stop and remove the containers from another terminal, run:

```

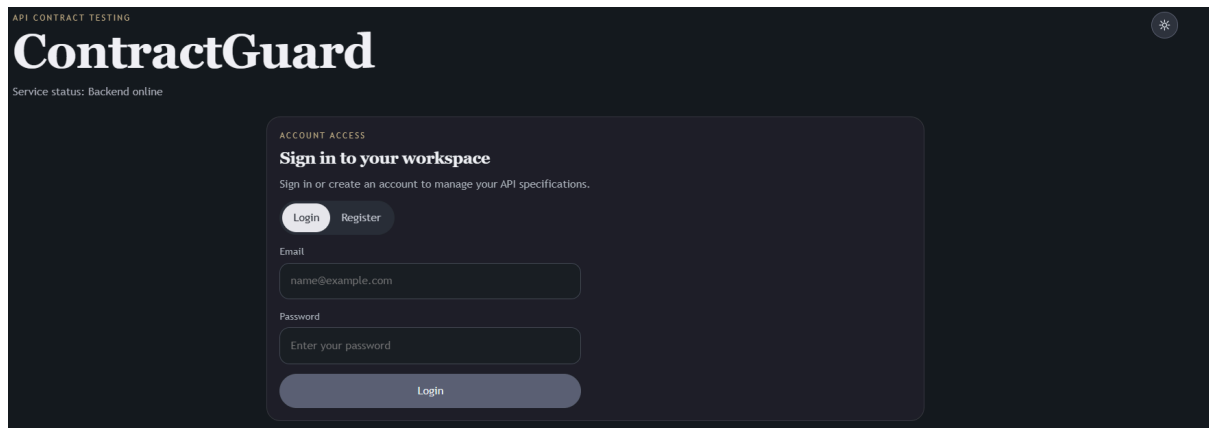
Shell
docker compose down

```


To remove the database volume as well, use:

```
Shell
docker compose down -v
```

Use -v only if you want to remove the stored PostgreSQL data created by the Docker setup.



First Launch and Account Creation

When the frontend is available, open the application in your browser. The first screen is the authentication panel. Use the Register tab to create a new account, then sign in.

The registration form requires:

- A valid email address
- A password

The backend rejects invalid email addresses and passwords that exceed 72 bytes.

Installation Verification Checklist

Before moving on, verify the following:

- The frontend opens in your browser.
- The backend health endpoint returns {"ok": true} (verified at <http://localhost:8000/api/health>).
- You can create an account and log in.
- The header shows ContractGuard and a service status value.

Installation Notes

- The backend creates its tables automatically on startup.
- Docker Compose is the supported release startup method for this application.

- The database container is managed automatically by Docker Compose.
- The current repository is configured for local development and testing. Production deployment hardening is outside the scope of this manual.

User Interface Overview

After you log in, ContractGuard opens a workspace-based interface built around two main views: Dashboard and Logs.

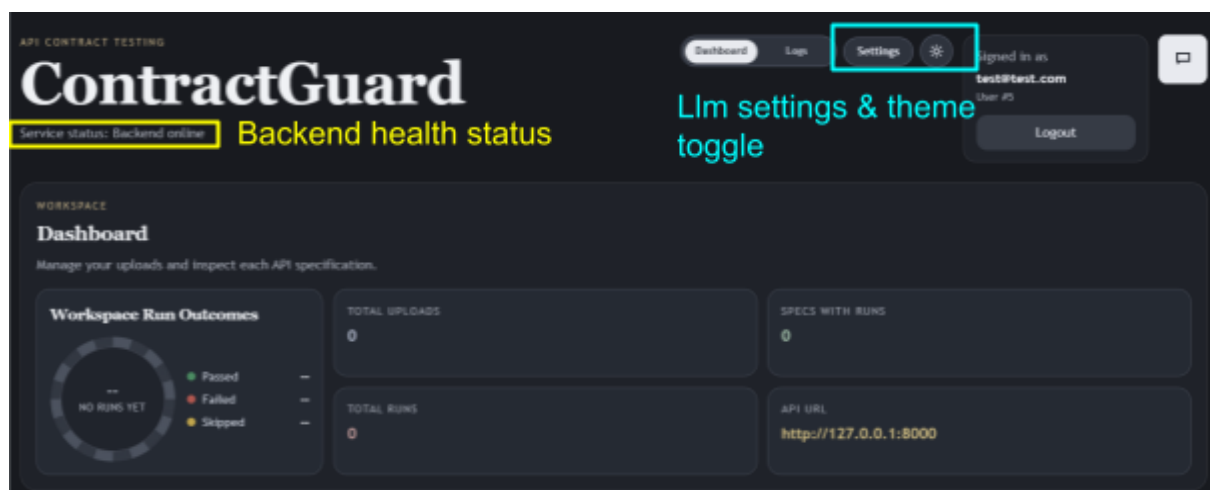
Header and Global Controls

The header appears at the top of the application and contains the primary global controls.

The header includes:

- The ContractGuard application title
- A service status indicator tied to backend health
- A segmented workspace toggle with Dashboard and Logs
- A Settings button for AI provider and prompt configuration
- A theme toggle
- Session controls, including the current signed-in user and logout access
- A chat toggle that opens or closes the assistant panel

The service status gives immediate confirmation that the frontend can reach the backend API. If the backend is unavailable, this is one of the first places where the problem is visible.



Dashboard View

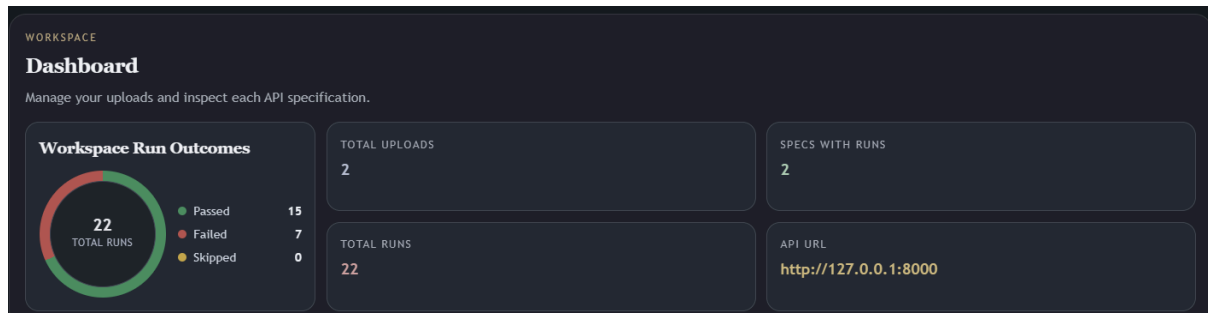
The Dashboard is the main working area for uploading specifications, reviewing generated tests, running those tests, and inspecting results.

The Dashboard contains the following core areas:

Workspace Summary

At the top of the main workspace, ContractGuard displays summary cards and an outcome visualisation. This area gives a quick overview of the currently stored workspace activity, such as how many uploads exist and how many recent results are passing or failing.

This section is most useful after at least one test run has been completed.



Upload Panel

The Upload panel is where specifications are imported into the application.

The panel supports:

- Selecting one or more files
- Selecting a folder
- File types .json, .yaml, and .yml

If you upload a folder, ContractGuard scans the selected files and imports only supported specification files. Unsupported files are skipped automatically.

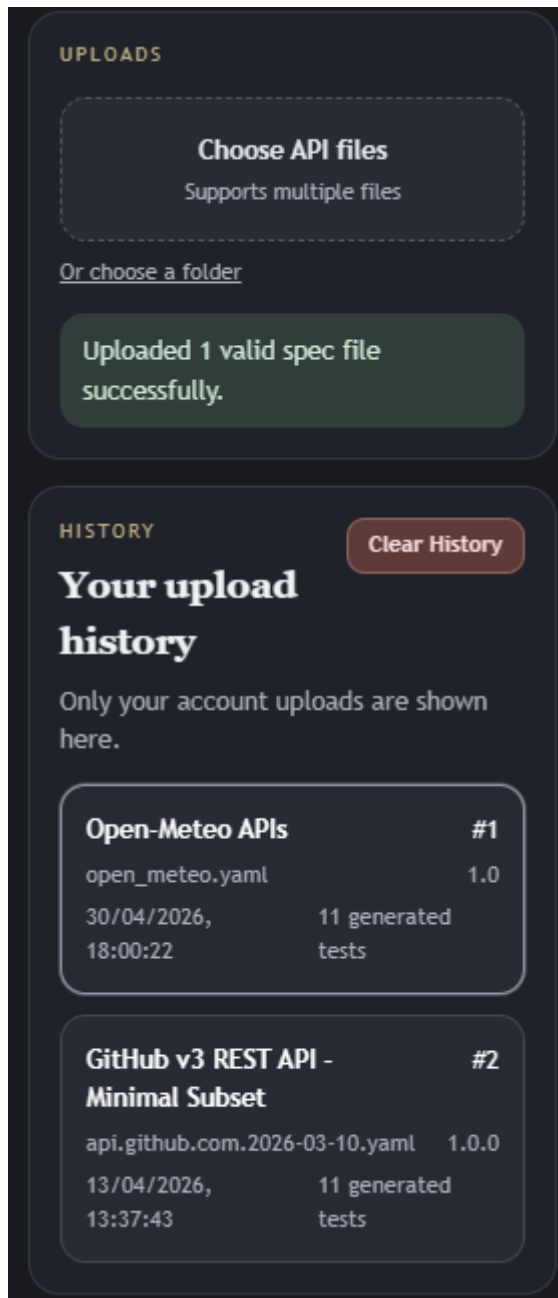
If you select several files and some are not valid OpenAPI files, ContractGuard uploads the valid files and skips the invalid ones.

Upload History Panel

The history panel lists specifications uploaded by the current account. Each history item shows:

- Title
- File name
- Version
- Upload time
- Generated test count when available

Selecting an item from history loads that specification into the details workspace. The Clear History action removes the uploaded specifications and their related stored artifacts.



Specification Details Workspace

When a history item is selected, the details workspace becomes the main operational area for that specification.

This area shows:

- Parsed API information
- Generated test coverage information
- Test categories
- Individual test case details

- Editable JSON input data per test case
- Expected result display
- Run controls
- Latest run summary

The details workspace is where most of the daily testing activity happens.

Test Coverage and Generated Cases

Generated tests are grouped into categories. Each category can be expanded to inspect the individual cases inside it. Each test case displays the HTTP method, path, title, action description, input data, expected result, and current run outcome when one exists.

Each case also includes:

- A Run button for rerunning only that test case
- A Reset action when the input JSON has been edited
- Failure details when the case fails
- AI actions when AI features are available

The screenshot displays the 'Test Coverage' interface. At the top, it shows 'Generated Test Preview' with a 'Total generated tests' count of 11. Below this, there are buttons for 'Run Tests' and 'Reset Defaults', and a text input field containing 'https://api.open-meteo.com'. A 'RUN AUTH' dropdown menu is set to 'None'. The 'Latest Run' section shows a summary: 'Total: 11', 'Passed: 11', 'Failed: 0', and 'Skipped: 0'. The main section is titled 'happy path' and shows a specific test case 'TC-GET-v1-forecast-001' with a status of 'PASS'. The test description is 'GET /v1/forecast - Happy path: minimal valid request returns 200'. The action is 'Send GET request to /v1/forecast with query params [latitude: 53.3498, longitude: -6.2603]'. The 'INPUT DATA' section shows a JSON object with path_params, query_params (latitude and longitude), headers, and body. The 'EXPECTED RESULT' section shows a JSON object with status_code 200 and a description. A 'Run' button is visible next to the test case.

Run Controls

At the top of the generated test preview area, ContractGuard shows the main run controls. These include:

- Run Tests for the full generated suite
- Reset Defaults for restoring the current run configuration and generated payloads to their default state
- A Base URL input
- A Run Auth selector with None, Bearer Token, and API Key Header
- Authentication fields shown only when the selected auth mode requires them

The Base URL field is required. If it is blank, the interface blocks test execution and displays an error.

If Bearer Token is selected, a token value is required.

If API Key Header is selected, an API key value is required, and the header name defaults to X-API-Key if no custom name is entered.

The current web interface does not expose a separate timeout field. Test runs use the application's current default per-test timeout, which is 10 seconds unless changed in code.

Settings Modal

The Settings button opens the settings modal. This modal has two functional areas:

1. Model configuration
2. Custom instruction configuration

The model configuration area allows you to:

- Select the active model
- Add a new model
- Provide a label for easier display
- Override the base URL when needed

Provider behavior is different depending on provider type:

- For OpenAI and Anthropic, an API key is required before ContractGuard can load provider models.
- For Ollama, you enter the model name directly and may optionally override the base URL.

The custom instruction area affects AI summaries for failed test cases. It does not affect suggested test generation.

ASSISTANT

LLM Settings

Close

Pick your model target and configure custom instruction behavior.

Model

Prompt

Choose your model

qwen3-coder:latest

Cancel

Add Model

Provider

OpenAI

API key

Required to load provider models

Model

Enter API key to load models

Label (optional)

Friendly display name

Base URL (optional)

Leave blank for provider default

Add Model

ASSISTANT

Dashboard

Users

Settings

Close

LLM Settings

Pick your model target and configure custom instruction behavior.

Model

Prompt

The prompts below are sent to the LLM when generating AI Summaries for failed test cases. Your custom instruction replaces the default system prompt for AI summaries when set. Suggested tests are not affected by custom instructions.

Default System Prompt

```
You are a contract-test failure analyst.
Analyze ONE API test case using ONLY the supplied evidence.
Do not describe framework structure, JSON fields, metadata names, or file paths.
Focus on the most likely failure mechanism for this exact case.
Distinguish confirmed facts from likely causes.
If this appears aligned with a negative/fail-path test intent, say so.
If evidence is incomplete, note the missing evidence briefly.
Output must be concise, practical, and machine-parseable JSON only.
```

User Prompt Template

```
Case:
- case_id: {case_id}
- intent/category: {intent}/{category}
- endpoint: {method} {path}
- expected: {expected_outcome}
- actual: {actual_outcome}

Evidence:
{evidence_json}

Task:
Give the best root-cause explanation for this case.
```

Custom Instruction

condense to only 1 sentence

Save Prompt

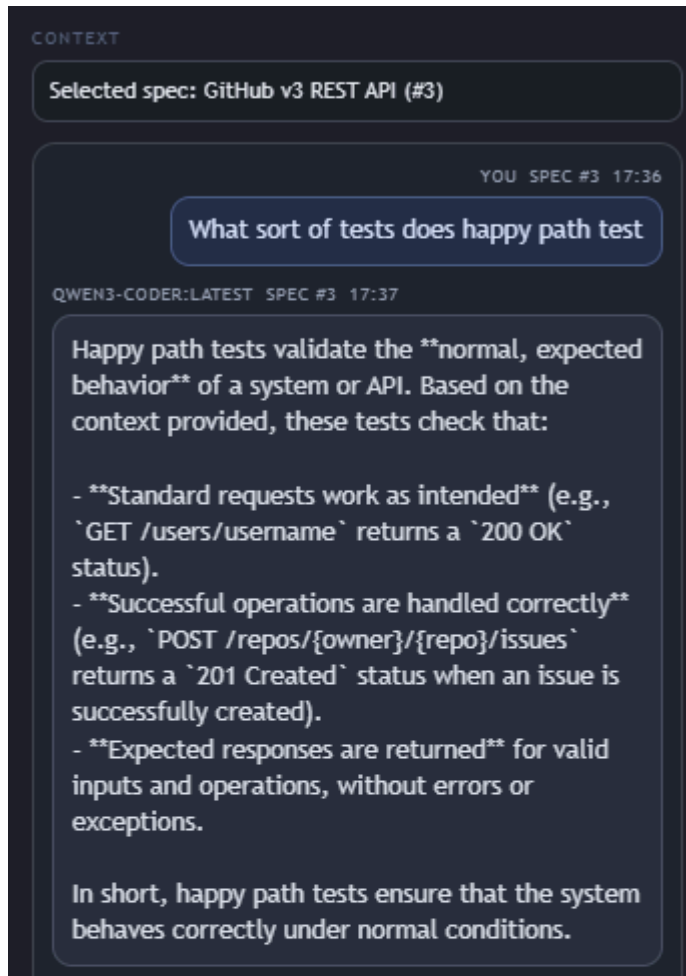
Reset

Chat Panel

The chat panel is a grounded assistant tied to the currently selected specification. It answers only within the context of:

- The selected specification
- The generated test cases
- The latest run information for that specification

If no specification is selected, chat is not fully active and prompts the user to select one first. The assistant does not serve as a general-purpose chatbot in this interface.



Logs View

The Logs workspace view is used to review saved run history.

Each run card in the Logs view shows:

- Specification title and version
- Run duration
- Base URL
- Total, passed, failed, and skipped counts
- Whether authentication was provided

- Whether AI outputs are available
- Recorded model names when available
- Run ID and run date

The Logs view also includes:

- A search field for specification title, file name, or version
- A filter for All Runs, Passed, or Failed
- A Clear Logs action

Clearing logs removes stored run records and saved AI insights, but it does not remove uploaded specifications.

The screenshot displays the ContractGuard application interface. At the top, the header includes the title "ContractGuard" and navigation links for "Dashboard", "Logs", and "Settings". A user status bar on the right indicates the user is signed in as "test0@test.com" with a "Logout" button. Below the header, the "WORKSPACE" section is titled "Logistics" and includes a "Clear Logs" button. A "Spec Search" field is provided for filtering by title, file, or version. A "Filter" dropdown menu is set to "All Runs", with options for "All Runs", "Passed", and "Failed". The main content area lists three specifications:

- GitHub v3 REST API** (Spec ID #6, Version 1.1.4, Duration 8.61 s, Base URL https://api.github.com). Summary: Total: 47, Passed: 32, Failed: 15, Skipped: 0. Auth: Provided (bearer). AI: Unavailable. No model recorded. Run ID #12, Run Date 30/04/2026, 16:45:32.
- Orders** (Spec ID #7, Version 2.25, Duration 26.6 s, Base URL https://api-m.sandbox.paypal.com). Summary: Total: 72, Passed: 13, Failed: 59, Skipped: 0. Auth: Provided (bearer). AI: Unavailable. No model recorded. Run ID #11, Run Date 30/04/2026, 16:45:19.
- Open-Meteo APIs** (Spec ID #8, Version 1.0, Duration 1.65 s, Base URL https://api.open-meteo.com). Summary: Total: 11, Passed: 11, Failed: 0, Skipped: 0. Auth: Not provided. AI: Unavailable. No model recorded. Run ID #10, Run Date 30/04/2026, 16:44:36.

At the bottom, it indicates "3 run logs shown" and "No more logs."

Run Detail and AI Detail Windows

Opening a stored run reveals more detailed information, including the saved suite, result rows, summary information, and related AI outputs.

Using ContractGuard

This section describes the normal end-to-end user workflow.

Sign In and Open the Workspace

1. Start the application using Docker Compose.
2. Open the frontend in your browser.

3. Register a new account if this is your first time using ContractGuard.
4. Sign in.

After login, the Dashboard opens automatically.

Upload an API Specification

To upload a specification:

1. In the Upload panel, click Choose API files to select one or more files, or use Or choose a folder to select a directory.
2. Choose one or more OpenAPI or Swagger files in JSON or YAML format.
3. Wait for the upload to finish.

When an upload succeeds, ContractGuard does the following automatically:

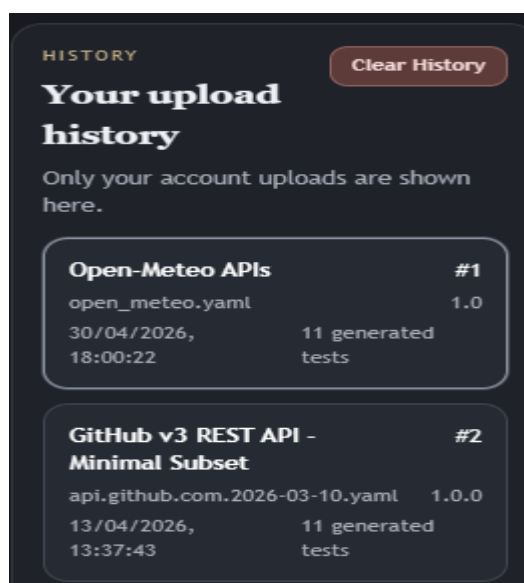
- Reads the file contents
- Parses the specification into an internal representation
- Generates a test suite from the parsed specification
- Saves the uploaded specification to your account history

If several files are selected, ContractGuard processes them individually. Valid specifications are saved. Files that are unsupported or invalid are skipped.

Select an Uploaded Specification

After upload, the new specification appears in Your upload history. Click the history entry to load it into the details workspace.

When selected, the workspace shows the specification title, version, generated tests, and run controls for that item.



Review the Parsed Specification

Before running tests, review the parsed API information. Confirm that:

- The specification title matches the expected API
- The version is correct
- The endpoints shown match the API you want to test
- The generated case count is reasonable for the uploaded contract

This step helps catch the wrong file, an outdated specification, or a malformed contract before test execution begins.

Review and Edit Generated Test Cases

ContractGuard generates test cases automatically from the uploaded contract. Expand the categories to inspect individual cases.

For each case, you can review:

- The test title
- The HTTP method and path
- The input data
- The expected result

The input JSON is editable. This allows you to refine generated case inputs before a run. If you make changes and want to discard them, use the case-level Reset action to restore the stored baseline version of that case.

Use this editing feature carefully. If you change test inputs significantly, the resulting run will reflect your modified payload rather than the originally generated default.



Configure a Full Test Run

Before starting a run, fill in the run configuration at the top of the generated test preview area.

Base URL

Enter the absolute base URL of the API implementation you want to test, for example:

None
https://api.example.com

This value is required. ContractGuard will not run a suite without it.

Authentication

Choose the appropriate run authentication mode:

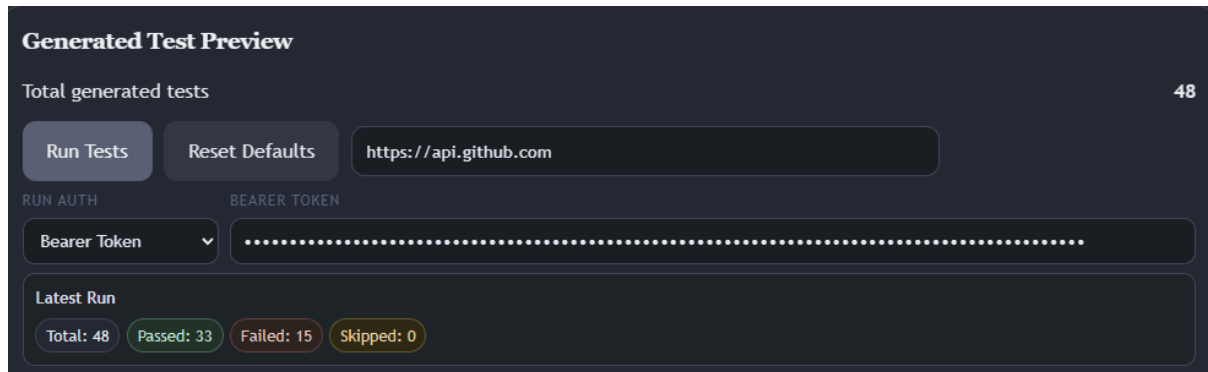
- None if the API does not require authentication
- Bearer Token if the API requires an Authorisation: Bearer ... header
- API Key Header if the API requires a custom header such as X-API-Key

If you choose Bearer Token, enter the token value.

If you choose API Key Header, enter:

- The API key value
- The header name, if different from X-API-Key

If the authentication mode requires a value and you leave it blank, ContractGuard blocks the run and displays a validation message.

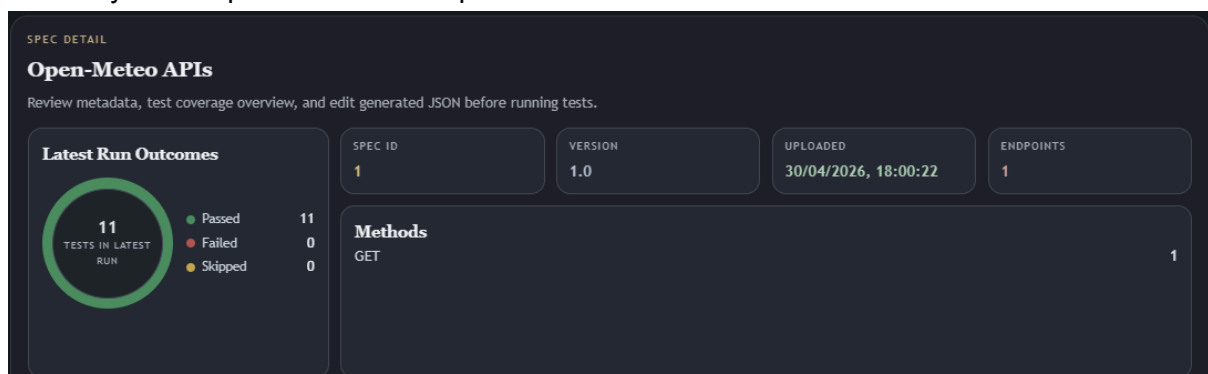


Run the Full Suite

To run the complete generated suite for the selected specification:

1. Confirm the base URL is present.
2. Confirm any required authentication fields are filled in.
3. Click Run Tests.

When the run completes, ContractGuard stores the result and displays the latest run summary in the specification workspace.



Run a Single Test Case

Each individual generated test case has its own Run button. Use this when you want to rerun only one test instead of the entire suite.

This is useful when:

- You edited only one input payload
- You fixed one endpoint on the API side and want to verify just that case
- You want to reproduce a failure quickly

When a single case is rerun, ContractGuard merges the new result back into the saved run for that specification. Any previously stored AI insights for that rerun test are removed so that future AI output matches the latest result.

Understand the Run Summary

After a run, ContractGuard shows a latest run summary that includes:

- Total
- Passed
- Failed
- Skipped

Use these counts as the first summary of suite health.

The meaning of each outcome is:

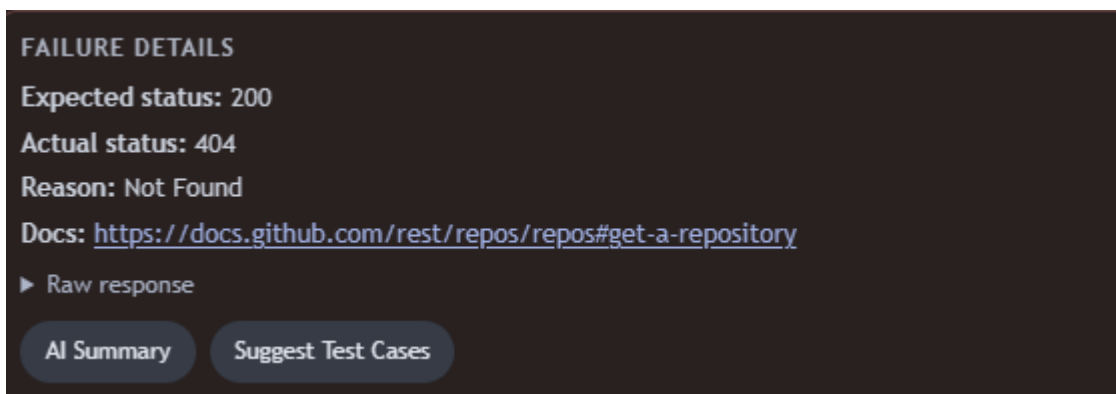
- PASS: The endpoint response matched the case expectations.
- FAIL: The request completed, but the response did not match the expected behavior.
- SKIP: The runner did not execute the case fully or could not complete it under the current conditions.

Inspect Failed Cases

Expand any test case with a failed result to see failure details. A failed case can display:

- Expected status
- Actual status
- A failure reason
- A documentation link, if one was extracted from the response details
- A raw response section when available

This information is the first place to diagnose contract mismatches, implementation bugs, missing authentication, or endpoint behavior that diverges from the specification.



Use AI Summary for Failed Cases

If an AI model is configured, a failed test case displays an AI Summary button.

Use AI Summary when you want ContractGuard to explain the failure in natural language using the available evidence from:

- The selected specification
- The generated test case
- The latest run result
- The configured model

If a custom instruction has been configured in Settings, that custom instruction is used for the AI summary flow.

After completion, ContractGuard displays:

- The explanation text
- Any warning or fallback message
- Optional LLM debug details when relevant
- The prompt snapshot sent to the model when available

If model output is unavailable or invalid, ContractGuard may use a deterministic fallback explanation.

Generate an Additional Test from a Failure

If an AI model is configured, a failed test case also displays a Suggest Test Cases button.

Use this action when you want ContractGuard to propose an additional follow-up test based on the failed case.

When a suggestion is returned, ContractGuard can display:

- A reason for the suggested extra test
- A warning or fallback notice
- The raw suggested test JSON
- An Add Suggested Test action if the generated case can be applied safely

When you click Add Suggested Test, the suggested case is added to the current generated test set in the workspace. Review it before running the suite again.

Custom instructions in Settings do not affect this suggested-test generation flow.

FAILURE DETAILS

Expected status: 200
 Actual status: 404
 Reason: Not Found
 Docs: <https://docs.github.com/rest/repos/repos#get-a-repository>

► Raw response

AI Summary Suggest Test Cases

AI EXPLANATION

The requested repository does not exist. The API returned a 404 status with a 'Not Found' message, indicating the repository 'standard-text/standard-text' was not found. Verify that the repository 'standard-text/standard-text' exists and is accessible with the provided credentials.

► Prompt sent to model

SUGGESTED EXTRA TEST

The test failed because repository 'standard-text/standard-text' does not exist on GitHub, returning 404. This is a happy path test expecting 200, so we need to use a repository that actually exists. The spec examples show 'octocat/Hello-World' as a valid public repository. Repairing the path_params to use this known existing repository should allow the test to pass.

► Suggested test JSON

Add Suggested Test GET /repos/{owner}/{repo} - Happy path: minimal valid request returns 200 has been added to payload

TC-GET-repos-owner-repo-001-repair GET /repos/{owner}/{repo} PASS

GET /repos/{owner}/{repo} - Happy path: minimal valid request returns 200 Run

Action: Send GET request to /repos/{owner}/{repo} using path params {"owner": "octocat", "repo": "Hello-World"}

INPUT DATA

```
{
  "path_params": {
    "owner": "octocat",
    "repo": "Hello-World"
  },
  "query_params": {},
  "headers": {},
  "body": null
}
```

EXPECTED RESULT

```
{
  "status_code": 200,
  "description": "Validate that GET /repos/{owner}/{repo} keeps 200 after repairing repo to a valid value."
}
```

Use the Grounded Chat Panel

Open the chat panel from the workspace and ask questions about the currently selected specification.

The chat assistant is designed for grounded support. Suitable questions include:

- Which endpoints were generated into tests?
- Why did a specific case fail?
- What does the latest run summary show?
- Which authentication style does this spec appear to use?

The assistant is intentionally limited to the selected specification, its generated tests, and the latest run context. It is not intended for unrelated questions.

Review Run History in Logs

Use the Logs view when you need to inspect previous runs across your workspace.

Typical uses include:

- Reopening a run after leaving the Dashboard
- Comparing recent failed and passed runs
- Checking which base URL was used for a run
- Reviewing saved AI outputs attached to that run

To review historical runs:

1. Click Logs in the workspace toggle.
2. Use Spec Search if needed.
3. Use the Filter dropdown to limit the view.
4. Click a run card to open its details.

The run details let you inspect saved results even after you move away from the original specification workspace.

ContractGuard

Service status: Backend online

WORKSPACE

Logistics

Run logs only. Uploaded specs remain after clearing logs.

Spec Search

Search by title, file, or version

Orders

Spec ID	Version	Duration
#6	2.25	28.9 s

Base URL: <https://api-m.sandbox.paypal.com>

Total: 72 Passed: 13 Failed: 59 Skipped: 0

Auth: Provided (Bearer) AI: Unavailable No model recorded

Open-Meteo APIs

Spec ID	Version	Duration
#1	1.0	1.72 s

Base URL: <https://api.open-meteo.com>

Total: 11 Passed: 11 Failed: 0 Skipped: 0

Auth: Not provided AI: Unavailable No model recorded

GitHub v3 REST API

Spec ID	Version
#6	1.1.4

Base URL: <https://api.github.com>

Total: 47 Passed: 32 Failed: 15 Skipped: 0

Auth: Provided (Bearer) AI: 1 explanation, 0 suggestions search codetodo

3 run logs shown

RUN DETAIL

Run #10

GitHub v3 REST API (spec #6)

UPLOADED: 30/04/2026, 16:20:15 RUN TIME: 30/04/2026, 16:20:28 DURATION: 9.01 s BASE URL: <https://api.github.com>

Auth: Provided (Bearer) Summary: 32 passed / 15 failed / 0 skipped AI records: 1

Test Case	Status	AI Records
GET /repos/{owner}/{repo} - Happy path: minimal valid request returns 200	FAIL	1 AI record
GET /repos/{owner}/{repo} - Negative: request non-existent resource	PASS	0 AI records
GET /repos/{owner}/{repo} - Error 404: non-existent owner	PASS	0 AI records
GET /repos/{owner}/{repo} - Error 404: non-existent repo	PASS	0 AI records
PATCH /repos/{owner}/{repo} - Happy path: minimal valid request returns 200	FAIL	0 AI records
PATCH /repos/{owner}/{repo} - Happy path: example-based request returns 200	FAIL	0 AI records
PATCH /repos/{owner}/{repo} - Negative invalid: body field 'visibility' - 'invalid_enum_value_xyz'	PASS	0 AI records
PATCH /repos/{owner}/{repo} - Negative invalid: body field 'squash_merge_commit_title' - 'invalid_enum_value_xyz'	PASS	0 AI records
PATCH /repos/{owner}/{repo} - Negative invalid: body field 'squash_merge_commit_message' - 'invalid_enum_value_xyz'	PASS	0 AI records
PATCH /repos/{owner}/{repo} - Negative invalid: body field 'merge_commit_title' - 'invalid_enum_value_xyz'	PASS	0 AI records
PATCH /repos/{owner}/{repo} - Negative invalid: body field 'merge_commit_message' - 'invalid_enum_value_xyz'	PASS	0 AI records

Clear All Logs

Use Clear Logs in the Logs view when you want to remove all stored run logs for your account while keeping uploaded specifications.

This action removes:

- Stored run records
- Saved AI insights tied to those runs

This action does not remove uploaded specifications.

Clear Upload History

Use Clear History in the upload history panel when you want to remove uploaded specifications permanently.

This action removes:

- Uploaded specification records
- Stored parsed artifacts
- Generated suite artifacts
- Associated test runs
- Associated AI insights tied to those specifications

Use this action with care because it is broader than Clear Logs.

Github Action Integration

Quick Start in Github

1. Create `.github/workflows/contractguard.yml` in your repo
2. Add GitHub Secrets (Settings - Secrets and variables - Actions)
3. Create `specs/` directory with your OpenAPI Specification
4. Create `config/runtime_mapping.json` with parameters
5. Push and create PR or specs in main branch changed - pipeline runs automatically

Step 1: Create Workflow File

Create `.github/workflows/contractguard.yml` under your `.github` workflow folder

```
None
name: ContractGuard API CI

on:
  push:
    branches:
```

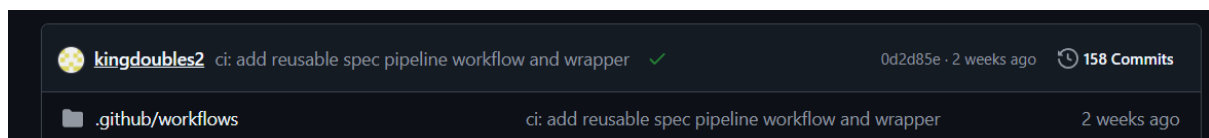
```

    - main
  paths:
    - "specs/**"
    - "config/**"
    - ".github/workflows/contractguard.yml"
  pull_request:
    paths:
      - "specs/**"
      - "config/**"
      - ".github/workflows/contractguard.yml"
  workflow_dispatch:

permissions:
  contents: read
  pull-requests: write

jobs:
  contractguard:
    uses:
kingdoubles2/FYP/.github/workflows/spec-pipeline-reusable.yml@Github_action_integration
  with:
    specs_path: specs
    runtime_mapping_path: config/runtime_mapping.json
    python_version: "3.11"
    suite_timeout_seconds: 12
    allow_suite_failure: false
    comment_on_pr: true
    engine_repository: kingdoubles2/FYP
    engine_ref: Github_action_integration
  secrets:
    CONTRACTGUARD_BEARER_TOKEN: ${ secrets.CONTRACTGUARD_BEARER_TOKEN
}}
    CONTRACTGUARD_API_KEY: ${ secrets.CONTRACTGUARD_API_KEY }}
    CONTRACTGUARD_API_KEY_HEADER: ${ {
secrets.CONTRACTGUARD_API_KEY_HEADER }}
    CONTRACTGUARD_AUTH_BY_BASE_URL_JSON: ${ {
secrets.CONTRACTGUARD_AUTH_BY_BASE_URL_JSON }}
    CONTRACTGUARD_RUNTIME_MAPPING_JSON: ${ {
secrets.CONTRACTGUARD_RUNTIME_MAPPING_JSON }}

```

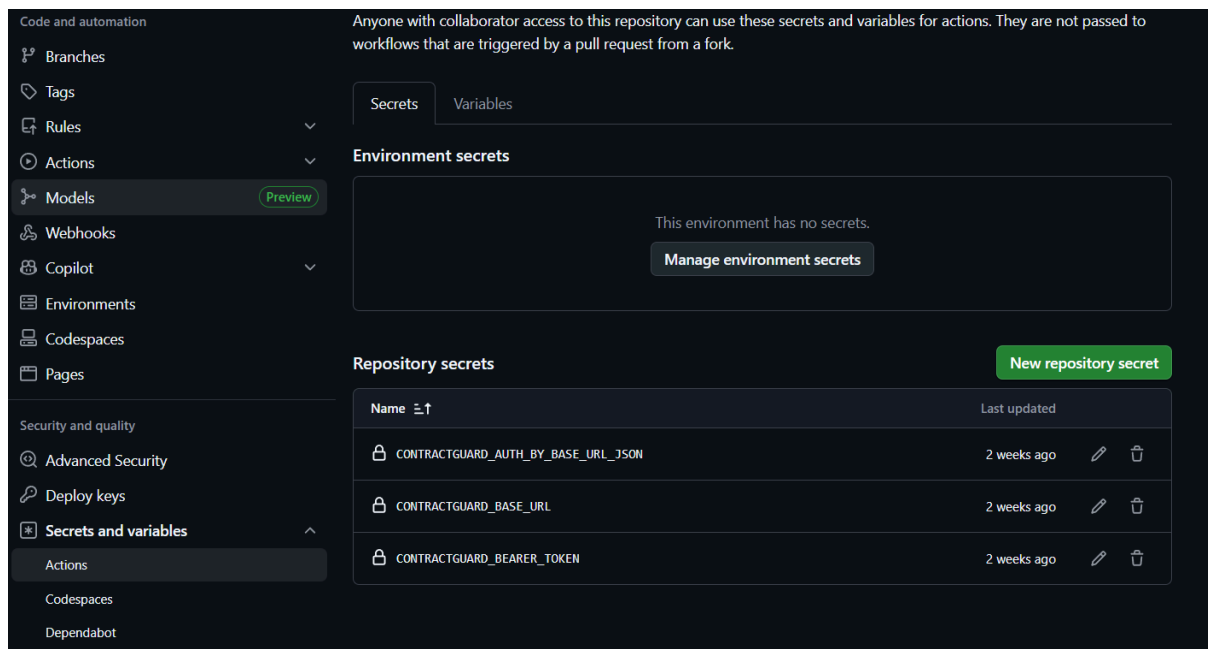


Step 2: Add GitHub Secrets

Go to: Your Repo - Settings - Secrets and variables - Actions - New repository secret

Common Secrets:

Name	Value	Example
CONTRACTGUARD_BEARER_TOKEN	Your API token	ghp_xxxxx (GitHub)
CONTRACTGUARD_API_KEY	API key	sk-xxxxx
CONTRACTGUARD_API_KEY_HEADER	Header name	X-API-Key
CONTRACTGUARD_AUTH_BY_BASE_URL_JSON	JSON for multi-API auth	JSON object for different API base URLs



For Multiple APIs with Different Auth:

Name: CONTRACTGUARD_AUTH_BY_BASE_URL_JSON

```
{
  "https://api.github.com": {
    "type": "bearer",
    "token": "ghp_xxx"
  },
  "https://api.openweathermap.org": {
    "type": "api_key",
    "key": "xxx",
    "header": "appid"
  }
}
```

Step 3: Repository Structure

your-repo/

— .github/workflows/contractguard.yml

— specs/

— api.yaml

— other-api.json

— config/
 — runtime_mapping.json

Step 4: Runtime Mapping File

Create runtime_mapping.json file

Example for Github related API

```
JSON
{
  "globals": {
    "owner": "your-username",
    "repo": "your-repo"
  },
  "suites": {
    "GitHub v3 REST API": {
      "owner": "your-username"
    }
  },
  "tests": {
    "TC-GET-users-username-001": {
      "username": "your-username"
    }
  }
}
```

Three levels (priority: global < suite < test):

- globals: All tests use these values
- suites: Specific test suite values override globals
- tests: Specific test case values override everything

Step 5: How Pipeline Works

Trigger:

Automatic: When you commit to specs/ in main folder or config

Manual: Actions tab - Run workflow

Jobs:

build-runtime-tests - Extracts IR, generates tests, injects mappings

run-suite-tests - Executes tests against real APIs, posts PR comment

Outputs (as Artifacts):

build/ir/ - Intermediate representation

build/tests/ - Generated test cases
 build/runtime-tests/ - Tests with mappings injected
 build/results/runtime-skip-report.json - Missing parameters

Workflow Configuration Options

Option	Default	Description
specs_path	src/specs	Where your OpenAPI specs are
runtime_mapping_path	runtime_mapping.json	Mapping file location
python_version	3.11	Python version
suite_timeout_seconds	12	HTTP request timeout
allow_suite_failure	false	Pipeline succeeds even if tests fail
comment_on_pr	true	Post results as PR comment
engine_repository	kingdoubles2/FYP/.github/workflows/spec-pipeline-reusable.yml@main	Engine repo
engine_ref	main	Pin to tag for reproducibility: v1.0.0

Example Configurations

GitHub API

Secrets:

CONTRACTGUARD_BEARER_TOKEN = ghp_xxxxx (Personal Access Token)

Mapping:

```
JSON
{
  "globals": {
    "owner": "username",
    "username": "username",
```

```

    "repo": "repo-name"
  }
}

```

PayPal Check out API

Secrets:

CONTRACTGUARD_BEARER_TOKEN = access token from PayPal

Mapping:

```

JSON
{
  "suites": {
    "PayPal Checkout API": {
      "order_id": "SAMPLE-ORDER",
      "client_id": "APXXXXXX"
    }
  }
}

```

Multi API (Different Auth per URL)

Secrets:

CONTRACTGUARD_AUTH_BY_BASE_URL_JSON below make it code block

```

JSON
{
  "https://api.github.com": {
    "type": "bearer",
    "token": "ghp_XXXXXX"
  },
  "https://api.stripe.com": {
    "type": "api_key",
    "key": "sk_live_XXXXXX",
    "header": "Authorization"
  },
  "https://api.openweathermap.org": {
    "type": "api_key",
    "key": "XXXXXX",

```



```

    "header": "appid"
  }
}

```

Viewing Results

Go to: Your repo - Actions

Click the workflow run (shows status)

The screenshot shows a GitHub Actions workflow run for the file `spec-min-ci.yml`. The workflow is titled `spec-min-ci.yml` and is triggered by `on: workflow_dispatch`. The workflow consists of three steps: `run-co... / build-runtime-tests` (25s), `run-contrac... / run-suite-tests` (45s), and `f... / summarize-and-comm...` (11s). The workflow status is **Success** and it took **1m 35s** to complete.

Annotations: 3 warnings. The warnings indicate that Node.js 20 actions are deprecated and will be forced to run with Node.js 24 by default. The warnings are for the following actions: `actions/checkout@v4`, `actions/setup-python@v5`, and `actions/upload-artifact@v4`.

Artifacts: Produced during runtime. The artifacts are listed in the following table:

Name	Size	Digest
<code>spec-pipeline-build</code>	1.8 MB	sha256:6fcfc5022c0e5c03e707ebf285cd83f771d60b6f2ecc47edc5dabbd14ab98ad
<code>spec-pipeline-summary</code>	675 Bytes	sha256:16e8d896a56aa9633911d1ac35f931ded1418afee717fa573635104257a04b0
<code>suite-results</code>	24.6 KB	sha256:5ef3b981372278881132f6554a8a5f41abf252a9c37d2f4e841be4b895a3fac

Check PR comment for test summary

Download "spec-pipeline-build" artifact to see detailed results

The screenshot shows a GitHub Pull Request page for the repository `kingdoubles2 / FYP`. The pull request is titled `Testing action with PR #6` and is created by `kingdoubles2`. The pull request is open and shows 29 commits, 3 checks, and 19 files changed.

The workflow run `ci: add reusable spec pipeline workflow and wrapper` is shown as successful. The workflow is titled `Spec Minimal CI` and is triggered by `on: pull_request`. The workflow consists of three steps: `run-contraction-guard-pipeline / build-runtime-tests`, `run-contraction-guard-pipeline / run-suite-tests`, and `run-contraction-guard-pipeline / summarize-and-comment`.

The **Download artifacts** section shows the following artifacts:

- `spec-pipeline-build` (1.8 MB)
- `spec-pipeline-summary` (675 Bytes)
- `suite-results` (25 KB)

Artifacts contain:

Test results (passed/failed/skipped counts)

Skip report (what parameters were missing)

Generated test cases in JSON format

Intermediate representations